

silent-agent-guard

Stop wasting tokens on events that should have stayed silent.

`silent-agent-guard` is a tiny, local-first Python guardrail for AI agents. It sits **before** your LLM call, filters low-signal events, and only escalates when a local threshold is crossed.

Most agent systems fail in the same boring way:

- They call an LLM for every small event.
- They summarize noise.
- They burn tokens during idle periods.
- They generate reports nobody reads.
- They drift into unsupervised action loops.
- They look “intelligent” while quietly becoming expensive.

`silent-agent-guard` gives your agent one missing primitive:

The right to stay silent.

The Problem: Agent Cost Overrun

Modern AI agents are too eager.

They wake up, inspect logs, call APIs, ask an LLM, generate summaries, produce markdown, notify humans, and repeat.

That pattern is acceptable for demos.
It is dangerous in production.

The real cost is not just tokens.

It is:

- unnecessary LLM calls
- noisy human alerts
- inflated cloud bills
- brittle automation loops
- false confidence from verbose output
- agent behavior nobody can audit

Most systems add observability after the cost is already spent.

`silent-agent-guard` blocks waste before the LLM is called.

The Silent Guard Philosophy

A production-grade agent should not think on every event.

It should first ask:

Is this event important enough to deserve an LLM call?

If the answer is no:

[STATUS_SILENT]

No cloud call.

No summary.

No notification.

No fake intelligence.

If the answer is yes:

[STATUS_TRIGGERED]

Generate a compact human-readable escalation card.

This simple local-first rule can reduce unnecessary LLM calls dramatically in event-heavy systems, especially when most events are low-signal.

What This Project Does

`silent-agent-guard` demonstrates a clean event gating pattern:

```
raw local event
→ local threshold filter
→ stay silent or fire
→ generate compact escalation card
→ only then call an LLM or notify a human
```

The public demo includes:

- zero external dependencies
- pure Python 3
- deterministic mock events
- local severity scoring
- metric threshold evaluation

- `[STATUS_SILENT]` output for low-signal events
 - a 28-line plain-text escalation card for critical events
-

Use Cases

Use it before:

- LLM workflow execution
- cron-based AI reports
- agent reflection loops
- log summarization
- alert generation
- browser automation
- trading research agents
- customer support triage
- devops incident assistants
- autonomous data pipelines

The goal is not to replace your agent framework.

The goal is to protect it.

How to Run

Download the repository and run:

```
python silent_guard.py
```

Or:

```
python3 silent_guard.py
```

No API key required.

No `.env` required.

No cloud account required.

Expected Output

The script injects two mock events:

1. An `info-only` event that should stay silent.
2. A `critical` event that should generate an escalation card.

Expected output:

```

=== silent-agent-guard demo ===

--- Case 1: info-only event ---
[STATUS_SILENT] event_name=heartbeat_check reason=below_local_threshold

--- Case 2: critical event ---
01 | [STATUS_TRIGGERED]
02 | event_name: payment_pipeline_error
03 | severity: critical
...
28 | next_action: escalate_to_llm_or_human_operator

```

The exact numbers may vary if you edit the mock data, but the behavior is fixed:

```

low signal → silence
critical signal → card

```

Core API

```

should_fire_llm(event: AgentEvent, policy: GuardPolicy) -> GuardDecision

```

Returns a deterministic decision:

- `fire=False` → stay silent
- `fire=True` → generate escalation card

Event Shape

All private business fields are intentionally removed.

The public event format is generic:

```

AgentEvent(
  event_name="payment_pipeline_error",
  severity="critical",
  metrics_snapshot={
    "error_count": 7,
    "latency_ms": 4200,
    "risk_score": 0.92,
  },
)

```

No vendor lock-in.
No provider SDK.
No private schema.

Why Local-First?

Because not every signal deserves a model.

A local rule engine is:

- cheaper
- faster
- deterministic
- auditable
- testable
- safer for unattended agents

LLMs should be reserved for interpretation, synthesis, and judgment.

They should not be used as a reflex.

Roadmap

Potential future directions:

- YAML policy file
 - JSONL event input
 - webhook mode
 - OpenTelemetry export
 - LangChain / CrewAI / AutoGen adapters
 - LiteLLM pre-call middleware
 - local SQLite event ledger
 - Docker image
 - dashboard mode
 - severity policy packs
-

Design Boundary

This project does **not**:

- call an LLM
- store API keys
- connect to brokers
- execute trades
- send private data
- require a SaaS dashboard

- mutate production systems

It only answers one question:

Should this event be allowed to wake the model?

Commercial Architecture Consulting

Built from real-world production agent operations.

If you are a company in Taiwan or anywhere globally and need help designing:

- local-first AI agent runtimes
- token-saving guardrails
- multi-agent workflow governance
- human approval gates
- browser automation safety layers
- cost-controlled LLM systems

you are welcome to contact the author for custom AI architecture consulting.

Bring your messy automation problem.

We will make the agent quiet, safe, and useful.

License

MIT

Author

Created by an independent AI systems architect building local-first, cost-aware agent runtimes from Taiwan.